AegisPay — Complete Implementation Roadmap

Audience: you, tomorrow morning, opening this file to start PHASE 0 -> STEP 1

Rule: BUILD -> TEST -> VERIFY -> MOVE FORWARD. Never implement a dependent module before its foundation exists.

Architecture invariant: AI reasons/recommends; the deterministic Control Plane (FastAPI + Postgres) owns money. The AI Runtime has no DB creds, no payment secrets, no Razorpay keys, no money tools. All financial actions flow Control Plane only.

Stack: FastAPI + PostgreSQL (RLS) + LangGraph + Razorpay test-mode. No extra microservices, no K8s/AWS/NPCI-prod until explicitly required.

0. HOW TO USE THIS ROADMAP

Each phase has exactly 11 sections:

1. Goal
2. Why now
3. Files to implement (existing = validate to Done, new = build)
4. What each file does
5. DB changes
6. APIs required
7. Depends on
8. Implementation steps (exact order)
9. Tests required
10. Acceptance criteria
11. What works after

Status tags used below:

• DONE — exists and is (largely) correct/tested.

• PARTIAL — logic exists but unwired / incomplete / not production-grade.

• TODO — not started.

1. CURRENT STATE AUDIT (honest)

Run `Get-ChildItem api -Recurse -File -Filter *.py | ?{$_.Length -gt 0}` showed ~70 non-empty `.py` files. Key facts:

Area	State	Notes
CI (api-ci / web-ci / integration)	DONE	`test` gate green.
`api/core/*` (security, jwt, authorization, rls, db, ratelimit, logging, observability, exceptions, config)	PARTIAL	Auth primitives exist + unit tests, but not fully wired to routers/RLS end-to-end.
`api/db/*` (models, session, repositories, init.sql)	PARTIAL	Tenant-pinned session + repos exist; must verify RLS actually enforced by migration.
`db/migrations/0001_initial.sql`, `db/seeds/dev_products.sql`	PARTIAL	Must reconcile with `api/db/models.py` (single source of truth).
`api/modules/payments/{state,provider}.py`	PARTIAL	Pure state machine + provider interface exist; no DB persistence / router.
`api/modules/commerce/{flow,safety}.py`	PARTIAL	`PurchaseFlow` + safety exist; not wired to DB/services.
`api/modules/policy/engine.py`	PARTIAL	Engine exists + tests; no risk/authorization hooks wired in.
`api/modules/protocol_gateway/{canonical,adapters,gateway}.py`	PARTIAL	Pure adapters exist + tests; no real transport.
`api/modules/{idempotency,refunds,budget,webhooks,reconciliation,audit,passport,events,outbox}`	PARTIAL	Pure logic exists; not wired.

Area	State	Notes
`api/services/{payments,razorpay,razorpay_mock}.py`	PARTIAL	Services exist; not called by routers.
`api/routers/*`	TODO (except health/me)	Only `health.py` + `router.py` (me) exist. **All domain routers missing.**
`api/ai_runtime/*`	TODO	`main.py`, `schemas.py` are stubs; agents/graph/prompts/tools empty.
`frontend/*`	TODO	`app/layout/page`, components, lib mostly stubs.
`workers/*`, `api/workers/*`	PARTIAL	Worker modules exist; not yet orchestrated by deploy/compose.
`tests/e2e`, `tests/redteam`, `tests/integration`	TODO	Only `test_health.py` integration; 40 unit tests only.

Conclusion: This is a partially-wired, logic-heavy scaffold. The roadmap therefore emphasizes (a) hardening + wiring existing logic, (b) building the missing routers/DB integration/AI tools/frontend, (c) integration + E2E + red-team proof.

2. DEPENDENCY GRAPH

Control-plane chain (owns money)

```
Phase 0 Foundation
   ↓
Phase 1 Database + RLS + Repositories
   ↓
Phase 2 Auth + Tenant Security (JWT, middleware, deps)
   ↓
Phase 3 Core API / App wiring (main, middleware, errors, routers registry)
   ↓
Phase 4 Catalog + Commerce (products, carts, orders)
   ↓
Phase 5 Policy → Risk → Authorization (decision engine)
   ↓
Phase 6 Payment Engine + Razorpay (state machine, idempotency, provider)
   ↓
Phase 7 Webhooks + Reconciliation (provider events, ledger match)
   ↓
Phase 8 Audit + Transaction Passport (immutable trail)
   ↓
Phase 16 Integration / E2E Testing
   ↓
Phase 18 Production Readiness
```

Async processing is a supporting dependency, not a late feature:

```
Phase 6 Payment + Phase 7 Webhook/Reconciliation contracts
   ↓
Phase 14 Workers / Async (run the already-tested handlers)
   ↓
Phase 16 Integration / E2E Testing
```

AI-runtime chain (isolated, no money)

```
Phase 9 AI Runtime + LangGraph (isolated deployable)
   ↓
Phase 10 GROW — AI Merchant (opportunities, campaigns via API)
   ↓
Phase 11 SELL — AI Buyer (tools call Control Plane via API, never DB)
   ↓
Phase 12 Protocol Gateway (ACP/UCP/AP2/A2A/MCP/x402 adapters over Control Plane API)
```

Cross-cutting (apply every phase)

```
Phase 15 Security + Red Team      (runs continuously; gates each phase)
Phase 17 Observability            (logging/metrics/tracing wired from Phase 3)
Phase 13 Frontend                 (consumes Control Plane APIs; can start after Phase 4)
```

3. PHASE STATUS TABLE

Phase	Title	Status	Headline work
0	Foundation	PARTIAL->DONE	tooling/CI/dev-env verify
1	Database	PARTIAL	reconcile migration↔models, enforce RLS

Phase	Title	Status	Headline work
2	Auth + Tenant	PARTIAL	wire JWT->deps->middleware->RLS
3	Core API	PARTIAL	app wiring, error model, router registry, observability base
4	Catalog + Commerce	PARTIAL	build routers+DB wiring for products/carts/orders
5	Policy+Risk+Authz	PARTIAL	wire engine + risk + authorization service
6	Payment + Razorpay	PARTIAL	wire state machine+provider+idempotency to routers
7	Webhooks + Recon	TODO/PARTIAL	processor+worker wiring
8	Audit + Passport	PARTIAL	ledger+passport wiring
9	AI Runtime	TODO	LangGraph agent + tool SDK (API-only)
10	GROW	PARTIAL	campaigns/opportunities wired
11	SELL / AI Buyer	TODO	buyer tools + flow
12	Protocol Gateway	PARTIAL	real transports
13	Frontend	TODO	Next.js app
14	Workers	PARTIAL	orchestrate workers in compose
15	Security + Red Team	TODO	adversarial test suite
16	Integration/E2E	TODO	cross-service tests
17	Observability	PARTIAL	metrics/tracing/dashboards
18	Prod Readiness	TODO	hardening, runbooks, demo

4. DETAILED PHASES

PHASE 0 — REPOSITORY / FOUNDATION

1. Goal: reproducible dev env, lint/type/test tooling, CI that fails red.

2. Why now: every later phase relies on `make dev`, `make lint`, `make test`, CI.

3. Files: `api/pyproject.toml`, `api/Makefile`, root `Makefile`, `.gitignore`, `api/.env.example`, `scripts/dev/bootstrap.{sh,ps1}`, `.github/workflows/{api-ci,web-ci,integration}.yml`, `README.md`, `STRUCTURE.md`, `CONTRIBUTING.md`, `.pre-commit-config.yaml`.

4. What each does: pyproject pins ruff/mypy/pytest; Makefile wraps common cmds; bootstrap spins Postgres+API via compose; CI runs lint+type+test; README/STRUCTURE document the system.

5. DB changes: none.

6. APIs: none.

7. Depends on: nothing.

8. Steps: 0.1 confirm pyproject has ruff/mypy/pytest configs -> 0.2 root+api Makefiles (`dev`,`lint`,`type`,`test`,`migrate`) -> 0.3 `.env.example` (DB_URL, JWT_SECRET, RAZORPAY_TEST_*, no secrets committed) -> 0.4 bootstrap scripts + `deploy/compose/docker-compose.dev.yml` -> 0.5 pre-commit (ruff, trailing-whitespace, end-of-file) -> 0.6 three CI workflows (api-ci required `test`) -> 0.7 README + STRUCTURE + CONTRIBUTING -> 0.8 `make lint && make type && make test` locally green.

9. Tests: config-level (pyproject valid; a trivial `test_smoke.py` that imports `api.main`).

10. Acceptance: `make dev` boots API+Postgres; CI is green on empty app; no secrets in repo.

11. Works after: anyone can clone, `make dev`, and run lint/type/test.

PHASE 1 — DATABASE (schema, RLS, repositories)

1. Goal: single source-of-truth schema with tenant isolation, plus repositories.

2. Why now: every module reads/writes here; RLS is the security backbone.

3. Files: `db/migrations/0001_initial.sql` (rewrite to match models), `db/seeds/dev_products.sql`, `api/db/models.py` (canonical), `api/db/session.py`, `api/db/repositories.py`, `api/core/db.py`, `api/core/rls.py`, `api/dependencies/db.py`, `db/init.sql`.

4. What each does: models = SQLAlchemy ORM; session = tenant-pinned `set_config('app.tenant_id', …)`; repositories = tenant-scoped CRUD per entity; rls = helpers/verification; db = engine/session factory; init.sql = role creation (`aegispay_app` non-superuser, no BYPASSRLS; `aegispay_migration` owner).

5. DB changes: tables — `tenants`, `users`, `products`, `carts`, `cart_items`, `orders`, `order_items`, `payments`, `payment_attempts`, `authorizations`, `campaigns`, `budget_ledger`, `webhooks`, `reconciliation`, `audit_entries`, `passport_records`, `outbox`, `idempotency_keys`, `opportunities`, `events`. Every tenant table has `tenant_id`. `ALTER TABLE … ENABLE ROW LEVEL SECURITY;` + policy `USING (tenant_id = current_setting('app.tenant_id')::uuid)`. `SET row_security = on;`. Indexes on `tenant_id`, FK cols, `order_id`, `payment_id`, `idempotency_key`.

6. APIs: none (internal).

7. Depends on: Phase 0.

8. Steps: 1.1 finalize `models.py` as source of truth -> 1.2 write `0001_initial.sql` to match exactly -> 1.3 add RLS policies + two roles + grants -> 1.4 `db/seeds/dev_products.sql` (sample tenant, merchant user, products) -> 1.5 `session.py` tenant pinning -> 1.6 `repositories.py` for each aggregate (OrderRepo, PaymentRepo, CampaignRepo w/ `atomic_reserve`) -> 1.7 `dependencies/db.py` yields pinned session -> 1.8 verify migration applies clean on fresh Postgres.

9. Tests: `tests/unit` + `tests/integration`: two tenants A/B; assert tenant B cannot read A's rows under RLS; assert repos enforce filter; assert `atomic_reserve` is transactional and safe under concurrency.

10. Acceptance: migration == models; RLS blocks cross-tenant at DB level (proven by test); repos green.

11. Works after: tenant-isolated persistence that cannot leak across tenants.

PHASE 2 — AUTHENTICATION + TENANT SECURITY

1. Goal: verifiable identity + tenant context on every request.

2. Why now: gating for all domain routers; RLS needs `app.tenant_id` set from a trusted claim.

3. Files: `api/core/jwt.py`, `api/core/security.py`, `api/core/authorization.py`, `api/core/ratelimit.py`, `api/dependencies/auth.py`, `api/middleware/middleware.py`, `api/schemas/common.py`, `api/tests/unit/test_jwt.py`, `test_authorization.py`, `test_ratelimit.py`.

4. What each does: jwt = sign/verify access+refresh (RS256/HS256, exp, tenant_id claim); security = password hash (argon2), API-key scheme; authorization = role/permission checks (`require_role`, `require_permission`); ratelimit = per-tenant token bucket; auth dependency = resolves `CurrentUser` + sets tenant context; middleware = request_id, tenant_context (sets `app.tenant_id` from verified token — never from frontend-supplied header), rate_limit, logging.

5. DB changes: `users` (password hash, roles, status), `api_keys` if used.

6. APIs: `POST /auth/login`, `POST /auth/refresh`, `GET /auth/me` (exists), `POST /auth/api-keys` (later).

7. Depends on: Phase 1.

8. Steps: 2.1 jwt sign/verify + tests -> 2.2 password hashing + login endpoint -> 2.3 `CurrentUser` dependency + role checks -> 2.4 middleware sets tenant context from token -> 2.5 rate limiting per tenant -> 2.6 refresh flow -> 2.7 authorization unit+integration tests (privilege escalation attempts fail).

9. Tests: token expiry/forgery rejection; tenant context cannot be spoofed via header; rate-limit enforcement; role checks.

10. Acceptance: unauthenticated -> 401; wrong tenant -> 403/empty; no tenant spoofing.

11. Works after: secure, tenant-scoped session identity.

PHASE 3 — CORE API / ARCHITECTURE WIRING

1. Goal: application shell that mounts routers, handles errors, emits observability.

2. Why now: domain phases plug into this.

3. Files: `api/main.py`, `api/routers/router.py`, `api/core/exceptions.py`, `api/core/logging.py`, `api/core/observability.py`, `api/middleware/middleware.py` (finalize), `api/schemas/common.py`.

4. What each does: main = FastAPI app, lifespan (engine connect/dispose), mounts routers, global exception handlers, CORS (strict), health; router = aggregator; exceptions = typed errors -> JSON problem+code; logging = structured JSON w/ request_id; observability = metrics/tracing hooks (Phase 17 fills).

5. DB changes: none.

6. APIs: `GET /health` (exists), `GET /health/ready`, `GET /health/live`.

7. Depends on: Phases 1–2.

8. Steps: 3.1 exception hierarchy + handlers -> 3.2 structured logging + request_id -> 3.3 CORS/security headers -> 3.4 lifespan + engine lifecycle -> 3.5 router registry (import placeholders for future routers) -> 3.6 `/health/ready` checks DB+RLS -> 3.7

integration test boots app via TestClient.

9. Tests: app boots; unhandled error -> 500 problem JSON w/ request_id; health probes correct.

10. Acceptance: consistent error envelope; every request logged with id; app importable & testable.

11. Works after: a real, observable API shell.

PHASE 4 — CATALOG + COMMERCE

1. Goal: products, carts, orders with tenant isolation, wired to DB.

2. Why now: required before policy/payment; commerce safety logic already exists.

3. Files: `api/modules/catalog/__init__.py` (+ `service.py`), `api/modules/commerce/flow.py` (extend), `api/modules/commerce/safety.py` (extend), `api/routers/{products,carts,orders}.py`, `api/schemas/{catalog,commerce}.py`, `api/repositories/` (OrderRepo, ProductRepo, CartRepo), `api/tests/{unit,integration}` for each.

4. What each does: catalog service = product CRUD (merchant-scoped); cart service = create/add/remove/clear, totals; order service = cart->order; flow = `PurchaseFlow` orchestrator using repos; safety = price/total sanity, no negative qty, integer paise.

5. DB changes: add any missing cols on `products/orders/carts` (verify against Phase 1).

6. APIs: `GET/POST/PUT /products`, `GET /products/{id}`, `POST /carts`, `POST /carts/{id}/items`, `DELETE /carts/{id}/items/{item}`, `POST /carts/{id}/checkout` -> order, `GET /orders/{id}`.

7. Depends on: Phases 1–3.

8. Steps: 4.1 product schema+repo+router -> 4.2 cart schema+repo+router -> 4.3 order schema+repo+router -> 4.4 `PurchaseFlow` wired to repos -> 4.5 commerce safety checks in endpoints -> 4.6 integration tests (cart->order happy path).

9. Tests: catalog tenant isolation; cart math; order creation; safety rejects bad input.

10. Acceptance: merchant can list products, build a cart, create an order — all tenant-scoped.

11. Works after: a catalog + cart + order backend (no payment yet).

PHASE 5 — POLICY + RISK + AUTHORIZATION

1. Goal: every financial action passes deterministic Policy -> Risk -> Authorization.

2. Why now: gates payments/refunds/campaigns; engine already exists (pure).

3. Files: `api/policy/engine.py` (finalize), `api/modules/risk/__init__.py` (+ `service.py`), `api/modules/authorization/__init__.py` (+ `service.py`), `api/modules/refunds/guard.py` (finalize), `api/routers/{policy,authorizations}.py`, `api/schemas/policy.py`.

4. What each does: engine = evaluate rules (amount limits, velocity, allow/deny/escalate); risk = score (amount, geo, new buyer, velocity) -> low/med/high; authorization = create/approve `Authorization` record (quorum for high-risk); refund guard = rules for refund eligibility.

5. DB changes: `authorizations` (status, required_approvals, approvers, policy_decision), `risk_scores` (optional).

6. APIs: `POST /policy/evaluate` (internal+limited), `POST /authorizations`, `GET /authorizations/{id}`, `POST /authorizations/{id}/approve`.

7. Depends on: Phase 4.

8. Steps: 5.1 engine finalize + integration tests -> 5.2 risk service + scoring -> 5.3 authorization service (create+approve) wired to engine/risk -> 5.4 refund guard wired -> 5.5 routers + schemas -> 5.6 adversarial tests (bypass attempts denied).

9. Tests: policy deny on limit; risk escalation; authorization quorum; refund guard; red-team: cannot skip authorization.

10. Acceptance: no financial action proceeds without a passing decision + (if required) approval.

11. Works after: a deterministic gate defending every money path.

PHASE 6 — PAYMENT ENGINE + RAZORPAY

1. Goal: complete test-mode Razorpay payment with idempotency + UNKNOWN handling.

2. Why now: core SELL capability; provider/state already exist (pure).

3. Files: `api/modules/payments/state.py` (finalize FSM), `api/modules/payments/provider.py` (+ `interface.py`), `api/services/razorpay.py`, `api/services/razorpay_mock.py`, `api/modules/idempotency/service.py` (finalize), `api/services/payments.py` (wire), `api/routers/payments.py`, `api/schemas/payments.py`, `api/routers/webhooks.py` (stub for Phase 7).

4. What each does:

• Payment state machine (`state.py`): `CREATED->AUTH_REQUIRED->AUTHORIZED->CAPTURED`, `FAILED`, `UNKNOWN`, `REFUNDED`, `PARTIALLY_REFUNDED`. Explicit transitions only.

• Provider interface (`provider.py:PaymentProvider`): `create_order`, `capture`, `refund`, `get_status` -> returns canonical status (never raw provider string trusted blindly).

• Razorpay impl (`razorpay.py`): calls test API with keys from env (Control Plane only).

• Mock provider (`razorpay_mock.py`): deterministic, injectable for tests/offline.

• Idempotency (`idempotency/service.py`): key -> same result on retry (DB-backed).

• Timeout/UNKNOWN handling: if provider call times out, mark `UNKNOWN`, schedule reconciliation; never assume success.

• Transaction handling: payment in DB transaction; outbox event emitted on state change.

• Authorization binding: authorization stores the immutable order/cart hash, amount, currency, and expiry; any material cart change invalidates the authorization before provider access.

• Provider boundary: the server derives amount and line items from its order snapshot; client/AI-supplied totals are never trusted.

• Service (`payments.py`): orchestrates FSM + provider + policy/authorization + idempotency + outbox.

5. DB changes: `payments`, `payment_attempts`, `idempotency_keys`, `outbox`.

6. APIs: `POST /payments` (create+authorize+capture with idempotency key), `GET /payments/{id}`, `POST /payments/{id}/refund`, `GET /payments/{id}/status`.

7. Depends on: Phases 4–5.

8. Steps: 6.1 FSM finalize + tests -> 6.2 provider interface -> 6.3 mock provider + tests -> 6.4 Razorpay impl (test keys) -> 6.5 immutable order/payment snapshot -> 6.6 authorization-to-snapshot binding and cart invalidation -> 6.7 idempotency finalize -> 6.8 timeout/UNKNOWN path -> 6.9 `payments.py` wiring (policy->risk->authz->provider->outbox) -> 6.10 routers + schemas -> 6.11 integration: full happy path with mock + with Razorpay test.

9. Tests: FSM illegal transitions rejected; client amount tampering rejected; cart mutation invalidates authorization; idempotency replay; timeout->UNKNOWN; refund guard; Razorpay test-mode capture.

10. Acceptance: a real Razorpay test-mode payment succeeds end-to-end, idempotent, auditable.

11. Works after: a complete, safe payment path.

PHASE 7 — WEBHOOKS + RECONCILIATION

1. Goal: ingest provider events, reconcile ledger, resolve UNKNOWN.

2. Why now: closes the loop on payments; required for correctness.

3. Files: `api/modules/webhooks/processor.py` (finalize), `api/modules/reconciliation/worker.py` (finalize), `api/workers/webhook_processor.py`, `api/workers/reconciliation_worker.py`, `api/routers/webhooks.py`, `api/schemas/webhooks.py`.

4. What each does: processor = verify signature, dedupe via idempotency, update FSM; reconciliation = periodic compare provider ledger vs local; auto-resolve UNKNOWN; alert on mismatch. Workers run the loops (Phase 14).

5. DB changes: `webhooks` (raw+processed), `reconciliation` (runs, diffs).

6. APIs: `POST /webhooks/razorpay` (signature-verified), `GET /reconciliation/status`.

7. Depends on: Phase 6.

8. Steps: 7.1 signature verification + dedupe -> 7.2 processor updates FSM -> 7.3 reconciliation diff logic -> 7.4 router + worker entrypoints -> 7.5 tests (duplicate webhook ignored; mismatch detected).

9. Tests: replay attack ignored; tampered signature rejected; reconciliation fixes UNKNOWN.

10. Acceptance: provider events safely update state; ledger reconciles.

11. Works after: payments stay correct despite async provider events.

PHASE 8 — AUDIT + TRANSACTION PASSPORT

1. Goal: immutable trail + portable proof per transaction.

2. Why now: trust/proof layer over everything above.

3. Files: `api/modules/audit/ledger.py` (finalize), `api/modules/passport/service.py` (finalize), `api/routers/{audit,passport}.py`, `api/schemas/{audit,passport}.py`.

4. What each does: ledger = append-only entries (hash-chained) for every state change; passport = assemble signed record (order, payment, authz, policy decision) -> verifiable receipt.

5. DB changes: `audit_entries` (hash chain), `passport_records`.

6. APIs: `GET /audit/entries?entity=`, `GET /passport/{payment_id}`.

7. Depends on: Phases 5–7.

8. Steps: 8.1 ledger append + hash chain verify -> 8.2 passport assembly + signature -> 8.3 routers -> 8.4 tests (tamper detection; passport verifies offline).

9. Tests: audit tamper detection; passport signature validates.

10. Acceptance: every action is provable and tamper-evident.

11. Works after: a verifiable audit + receipt system.

PHASE 9 — AI RUNTIME + LANGGRAPH (isolated)

1. Goal: a separate deployable that reasons/recommends via API tools only.

2. Why now: SELL/GROW/Protocol phases depend on it; must remain isolated.

3. Files: `api/ai_runtime/main.py`, `schemas.py`, `agents/__init__.py`, `graph/__init__.py`, `prompts/__init__.py`, `tools/__init__.py` (+ `client.py`, `tool_defs.py`), `Dockerfile.ai`, `deploy/compose/docker-compose.yml` (separate service).

4. What each does: main = FastAPI/LangGraph app; client = HTTP client to Control Plane API only using short-lived, audience-restricted, least-privilege agent credentials (not a broad merchant API key; no DB, no Razorpay secret); tools = discover products, create cart, request authorization, check status (all via an explicit allowlist); graph = intent->plan->tool-call loop; prompts = system prompts enforcing "recommend, never directly pay."

5. DB changes: none (AI has no DB).

6. APIs (AI Runtime exposes): `POST /agent/run` (intent in, structured plan/actions out).

7. Depends on: Phase 3 (Control Plane API surface) — AI only calls those APIs.

8. Steps: 9.1 define AI↔Control-Plane API contract -> 9.2 define scoped agent credential and audience checks -> 9.3 `client.py` (typed, allowlisted HTTP calls only) -> 9.4 tool definitions (read-only + request actions; no capture/refund tool) -> 9.5 LangGraph graph (reason->recommend->request) -> 9.6 prompts enforcing guardrails -> 9.7 `Dockerfile.ai` + compose service with no DB/payment env and restricted network egress -> 9.8 unit tests on graph (no money action taken directly).

9. Tests: AI never calls DB; AI cannot capture payment; AI only requests authorization.

10. Acceptance: isolated runtime that can reason and request, but cannot move money itself.

11. Works after: a safe, isolated AI brain.

PHASE 10 — GROW (AI Merchant)

1. Goal: opportunities, upsell/cross-sell, campaigns with budget control.

2. Why now: merchant revenue growth; budget logic exists (pure).

3. Files: `api/modules/opportunities/__init__.py` (+ `service.py`), `api/modules/campaigns/budget.py` (finalize), `api/modules/campaigns/__init__.py` (+ `service.py`), `api/routers/{opportunities,campaigns}.py`, `api/schemas/{opportunities,campaigns}.py`.

4. What each does: opportunities = generate growth suggestions (rules/LLM recommend, deterministic persist); campaigns = create with `budget_cap`; budget service = `atomic_reserve` spend, block overspend; safety = no negative budget.

5. DB changes: `campaigns`, `budget_ledger`, `opportunities`.

6. APIs: `POST /opportunities/generate`, `GET /opportunities`, `POST /campaigns`, `POST /campaigns/{id}/reserve`, `GET /campaigns/{id}/budget`.

7. Depends on: Phase 4 (products/orders) + Phase 9 (AI recommends).

8. Steps: 10.1 opportunity service -> 10.2 campaign + budget finalize -> 10.3 atomic reserve + tests -> 10.4 routers -> 10.5 integration: overspend blocked.

9. Tests: budget cap enforced; concurrent reserves safe; opportunity tenant-scoped.

10. Acceptance: merchants get AI growth ideas + capped campaigns.

11. Works after: GROW loop functional.

PHASE 11 — SELL / AI BUYER

1. Goal: AI buyer discovers, carts, authorizes, pays — all via Control Plane.

2. Why now: the headline SELL capability.

3. Files: `api/ai_runtime/tools/*` (buyer tools), `api/ai_runtime/graph/*` (buyer intent), `api/tests/integration` AI->API flows.

4. What each does: buyer tools call Control Plane: search products, add to cart, `POST /carts/{id}/checkout`, `POST /authorizations` (request), `POST /payments` (with returned authz). Graph enforces: never capture without authorization id.

5. DB changes: none (AI has none).

6. APIs reused: Phase 4 + 5 + 6 endpoints.

7. Depends on: Phase 9 + Phases 4–6.

8. Steps: 11.1 buyer tool set -> 11.2 buyer graph (discover->cart->authz->pay) -> 11.3 guardrail tests -> 11.4 end-to-end AI buyer run (mock provider) -> 11.5 with Razorpay test.

9. Tests: AI buyer completes purchase only with authorization; cannot bypass.

10. Acceptance: a full AI-driven purchase in test mode.

11. Works after: SELL end-to-end by AI.

PHASE 12 — PROTOCOL GATEWAY

1. Goal: ACP/UCP/AP2/A2A/MCP/x402/NPCI-ready adapters over Control Plane API.

2. Why now: interop; canonical model exists (pure).

3. Files: `api/modules/protocol_gateway/{canonical,adapters,gateway}.py` (finalize + real transports), `api/routers/protocol.py`, `api/schemas/protocol.py`.

4. What each does: canonical = normalized intent/order/pay objects; adapters = translate each protocol -> canonical -> Control Plane call; gateway = route inbound protocol messages, never expose DB/money directly.

5. DB changes: none (stateless translation).

6. APIs: `POST /protocol/{acp|ucp|ap2|a2a|mcp|x402}` (translate+delegate).

7. Depends on: Phases 4–6 (targets) + Phase 9 (agents).

8. Steps: 12.1 canonical finalize + tests -> 12.2 one adapter at a time (MCP first, then A2A, ACP, UCP, AP2, x402) -> 12.3 gateway routing + auth -> 12.4 NPCI/UPI readiness (interface only, no prod creds) -> 12.5 tests per adapter.

9. Tests: each adapter round-trips canonical↔protocol; invalid msg rejected.

10. Acceptance: external agents can transact via protocols through the gateway.

11. Works after: multi-protocol interoperability.

PHASE 13 — FRONTEND

1. Goal: merchant + operator UI consuming Control Plane APIs.

2. Why now: usable product; can start after Phase 4.

3. Files: `frontend/src/app/*`, `src/components/*`, `src/lib/api.ts`, `src/lib/auth.ts`, configs.

4. What each does: pages for login, products, carts, orders, payments, authorizations, campaigns, audit/passport, dashboard; `api.ts` typed client; auth token handling (no trust of tenant from UI).

5. DB changes: none.

6. APIs consumed: all Control Plane endpoints.

7. Depends on: Phases 2–8 (for endpoints).

8. Steps: 13.1 scaffold + auth flow -> 13.2 products/carts/orders UI -> 13.3 payments/authorizations UI -> 13.4 campaigns/opportunities -> 13.5 audit/passport viewer -> 13.6 E2E (Playwright) smoke.

9. Tests: Playwright checkout smoke; auth/tenant correct.

10. Acceptance: a human can do everything the API can, safely.

11. Works after: usable product UI.

PHASE 14 — WORKERS + ASYNC PROCESSING

1. Goal: reliable background jobs.

2. Why now: handlers are defined in Phases 6–8, but production correctness requires them to run reliably and idempotently in the background.

3. Files: `api/workers/{outbox_relay,reconciliation_worker,webhook_processor}.py`, `workers/*`, `deploy/compose` wiring, `scripts/ci/run-integration.sh`.

4. What each does: outbox_relay = publish outbox events; recon_worker = scheduled reconcile; webhook_processor = consume queue; hold-expiry = release expired authorizations.

5. DB changes: none new (uses outbox/reconciliation tables).

6. APIs: internal/health only.

7. Depends on: Phase 6 payment/outbox contracts and Phase 7 webhook/reconciliation handlers; Phase 8 audit events should be emitted by those handlers.

8. Steps: 14.1 outbox relay contract -> 14.2 recon worker cron -> 14.3 webhook worker -> 14.4 hold-expiry -> 14.5 retry/backoff and dead-letter behavior -> 14.6 compose orchestration with health checks -> 14.7 integration test.

9. Tests: outbox delivered; expired hold released; recon runs.

10. Acceptance: async work happens reliably.

11. Works after: resilient async backend.

PHASE 15 — SECURITY + RED TEAM

1. Goal: prove the guardrails hold under attack.

2. Why now: continuous; gates release.

3. Files: `api/tests/redteam/*`, `docs/GUARDRAILS_AND_SAFETY.md` updates.

4. What each does: adversarial tests — tenant spoofing, authz bypass, negative amounts, idempotency replay, webhook forgery, AI-money-direct attempts, RLS escape.

5. DB changes: none.

6. APIs: all.

7. Depends on: all prior.

8. Steps: 15.1 red-team harness -> 15.2 per-phase attacks -> 15.3 fix + re-test -> 15.4 document guardrails.

9. Tests: every attack in the matrix fails safely.

10. Acceptance: no critical finding open.

11. Works after: demonstrably hardened system.

PHASE 16 — INTEGRATION / E2E TESTING

1. Goal: cross-service proof.

2. Why now: validates the whole.

3. Files: `tests/e2e/playwright.config.ts`, `specs/checkout-flow.spec.ts`, `api/tests/integration/*` (full chains), `scripts/ci/run-integration.sh`.

4. What each does: API-level chain (auth->catalog->cart->policy->authz->pay->webhook->recon->audit) + UI smoke.

5. DB changes: none.

6. APIs: all.

7. Depends on: Phases 4–14.

8. Steps: 16.1 API E2E chain -> 16.2 UI E2E -> 16.3 CI integration job -> 16.4 all green.

9. Tests: full chain + UI.

10. Acceptance: one command proves the system end-to-end.

11. Works after: shippable confidence.

PHASE 17 — OBSERVABILITY

1. Goal: see everything in prod.

2. Why now: needed before real users.

3. Files: `api/core/observability.py` (finalize), `deploy/docker/nginx.conf`, dashboard configs, `api/core/logging.py` enrich.

4. What each does: metrics (prometheus), tracing (request_id span), structured logs, dashboards for payment funnel + RLS violations.

5. DB changes: none.

6. APIs: `/metrics`.

7. Depends on: Phase 3.

8. Steps: 17.1 metrics -> 17.2 tracing -> 17.3 dashboards -> 17.4 alert on payment failures/RLS anomalies.

9. Tests: `/metrics` emits; trace ids propagate.

10. Acceptance: operators can observe + alert.

11. Works after: production-observable system.

PHASE 18 — PRODUCTION READINESS

1. Goal: harden, document, demo.

2. Why now: final gate.

3. Files: `README.md`, `docs/ARCHITECTURE.md`, `API_SPEC.md`, `DEVELOPMENT_LOG.md`, runbooks, `deploy/compose/docker-compose.yml` prod-like, secrets handling docs.

4. What each does: final security pass, runbooks, demo script, documentation completeness.

5. DB changes: none.

6. APIs: all documented in OpenAPI.

7. Depends on: all.

8. Steps: 18.1 OpenAPI complete -> 18.2 runbooks -> 18.3 final red-team + load smoke -> 18.4 demo rehearsal -> 18.5 sign-off.

9. Tests: full suite green; demo runs.

10. Acceptance: Definition of Done met for all phases.

11. Works after: AegisPay is complete and demo-ready.

5. FILE-BY-FILE IMPLEMENTATION ORDER (build sequence)

```
Phase 0 : pyproject → Makefiles → .env.example → compose.dev → pre-commit → CI → README/STRUCTURE/CONTRIBUTING
Phase 1 : models.py → 0001_initial.sql → init.sql(roles) → session.py → repositories.py → dependencies/db.py → db tests
Phase 2 : jwt.py → security.py → dependencies/auth.py → authorization.py → middleware.py → ratelimit.py → auth tests
Phase 3 : exceptions.py → logging.py → observability.py(base) → main.py → router.py → health probes → app tests
Phase 4 : catalog(service/repo/router) → carts(service/repo/router) → orders(service/repo/router) → commerce/flow+safety wir
Phase 5 : policy/engine.py(final) → risk/service.py → authorization/service.py → refunds/guard.py(final) → routers → tests
Phase 6 : payments/state.py(final) → payments/provider.py(interface) → razorpay_mock.py → razorpay.py → idempotency(final) -
Phase 7 : webhooks/processor.py(final) → reconciliation/worker.py(final) → routers/webhooks.py → workers entry → tests
Phase 8 : audit/ledger.py(final) → passport/service.py(final) → routers → tests
Phase 9 : ai_runtime/client.py → tools/ → graph/ → prompts/ → main.py → Dockerfile.ai → compose → tests
Phase 10: opportunities/service.py → campaigns/service.py(+budget final) → routers → tests
Phase 11: ai_runtime buyer tools → buyer graph → E2E AI run
Phase 12: protocol_gateway canonical(final) → adapters(one by one) → gateway → routers/protocol.py → tests
Phase 13: frontend auth → products/carts/orders → payments/authz → campaigns → audit → E2E
Phase 14: workers outbox_relay → recon_worker → webhook_processor → hold-expiry → compose
Phase 15: redteam harness → per-phase attacks → fixes
Phase 16: api E2E chain → ui E2E → CI integration
Phase 17: metrics → tracing → dashboards → alerts
Phase 18: openapi → runbooks → final pass → demo
```

6. MILESTONES (MVP1–10)

MVP	Reaches at end of	What you can DEMO
MVP1	Phase 1	`make dev` boots API + Postgres; `GET /health` green; tenant-isolated DB proven by test.
MVP2	Phase 2–3	Login -> JWT; `/auth/me`; tenant context set from token; structured logs; 401/403 enforced.
MVP3	Phase 4–5	Merchant UI/API: list products, build cart, create order; policy+risk+authorization gate shown denying/approving.
MVP4	Phase 6	**Live Razorpay test-mode payment** through the API: cart->authz->pay->captured, idempotent, auditable.
MVP5	Phase 7–8	Webhook ingestion + reconciliation fixes UNKNOWN; audit ledger + transaction passport receipt downloadable.

MVP	Reaches at end of	What you can DEMO
MVP6	Phase 9–11	**AI buyer end-to-end**: "buy X" -> AI discovers, carts, requests authorization, pays in test mode. AI never touches money directly.
MVP7	Phase 10	GROW: AI suggests opportunities; merchant launches capped campaign; budget never overspends.
MVP8	Phase 12	Protocol Gateway: an external MCP/A2A agent completes a purchase through the gateway.
MVP9	Phase 13–14	Full frontend (merchant + operator) + workers running in compose; async outbox/recon/hold-expiry operational.
MVP10	Phase 15–18	Red-team proof (guardrails hold), observability dashboards, full E2E green, documented runbooks, final demo.

7. TESTING STRATEGY PER PHASE

• Phase 0–1: migration↔models parity; RLS cross-tenant negative tests; repo concurrency (`atomic_reserve`).

• Phase 2: JWT forgery/expiry; tenant-spoof via header rejected; rate-limit; role checks.

• Phase 3: app boot; error envelope; health probes.

• Phase 4: catalog isolation; cart math; order creation; safety rejects.

• Phase 5: policy deny; risk escalation; authz quorum; refund guard; bypass attempts denied.

• Phase 6: FSM illegal transitions; idempotency replay; timeout->UNKNOWN; Razorpay test capture; refund guard.

• Phase 7: duplicate webhook ignored; tampered signature rejected; recon fixes UNKNOWN.

• Phase 8: audit tamper detection; passport verifies offline.

• Phase 9: AI never reaches DB/Razorpay; only requests authz.

• Phase 10: budget cap; concurrent reserve safety.

• Phase 11: AI buyer completes only with authz; cannot bypass.

• Phase 12: each adapter round-trips; invalid msg rejected.

• Phase 13: Playwright checkout smoke; auth/tenant correct.

• Phase 14: outbox delivered; expired hold released; recon runs.

• Phase 15: red-team matrix — all attacks fail safely.

• Phase 16: full API+UI chain in CI.

• Phase 17: `/metrics` emits; trace ids propagate.

• Phase 18: full suite + demo rehearsal.

Run order each phase: `make lint && make type && make test` then (if integration) `make migrate && run integration`.

8. FINAL END-TO-END IMPLEMENTATION ORDER

`0 → 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12 → 13 → 14 → 15 → 16 → 17 → 18`

With frontend (13) allowed to begin in parallel after Phase 4, and observability (17) baseline wired at Phase 3 then enriched at 17. Red-team (15) runs continuously from Phase 4 onward, formalized at 15.

9. THINGS NOT TO BUILD YET (explicit NON-GOALS)

• No production Razorpay live keys / real money movement — test mode only.

• No NPCI/UPI production integration — interface/readiness only.

• No K8s, AWS, GCP, Terraform, service mesh — single compose deploy is enough.

• No extra microservices — Control Plane + AI Runtime are the only two units.

• No new databases / Redis cluster / Kafka — Postgres + outbox table suffice.

• No merchant PSP beyond Razorpay (until gateway demands it).

• No ML model training / vector DB — use LLM API calls + deterministic rules.

• No GraphQL — REST + typed schemas only.

• No frontend framework other than Next.js as chosen.

• No "AI autonomously pays" — ever. AI only requests; Control Plane decides.

• No trusting the frontend for tenant identity — token/RLS only.

10. SENIOR ENGINEERING REVIEW NOTES

The roadmap was reviewed against the repository structure and current implementation state.

The following corrections were made before export:

• Async dependency clarified: webhook and reconciliation handlers are designed in Phases 6–8, while Phase 14 provides their reliable worker execution. Phase 14 must finish before end-to-end sign-off.

• Payment integrity strengthened: payment execution now requires an immutable order/cart snapshot, authorization binding, server-derived amounts, and invalidation after material cart changes.

• AI isolation strengthened: the AI Runtime uses short-lived, audience-restricted credentials and an explicit tool allowlist. It receives no broad merchant credential, database credential, payment secret, or capture/refund tool.

• GROW/SELL ordering corrected: GROW precedes SELL in the AI capability chain because it depends on the shared runtime and catalog, but neither can bypass the control plane.

• Scope kept practical: no new microservices, production PSP credentials, protocol compliance claims, or infrastructure expansion are included in the build sequence.

Residual risk to resolve during implementation: the roadmap names the required security boundaries, but only integration and red-team tests can prove that the running deployment enforces them.

11. DEFINITION OF DONE (per phase)

A phase is complete only when ALL hold:

• [ ] implementation exists (no TODO stubs in shipped paths)

• [ ] unit + required integration tests exist and pass (`make test` green)

• [ ] API contracts updated in `docs/openapi/openapi.yaml` (if APIs added)

• [ ] documentation updated where necessary (README/STRUCTURE/docs)

• [ ] security implication handled (RLS/tenant/red-team where relevant)

• [ ] acceptance criteria satisfied

• [ ] CI (api-ci) green including the phase's tests

Start tomorrow at `PHASE 0 -> STEP 0.1`. Each step is independently verifiable. Do not skip the testing step of any phase.